Beyond Merge Rates: Measuring Abandoned Pull Requests from AI Coding Agents

Marten Weijer
Dept. of Computer Information Science
Minnesota State University, Mankato
Mankato, Minnesota
marten.weijer@mnsu.edu

***Abstract*—Merge rates are often used to judge AI coding agents, but an unmerged pull request alone does not show that a person rejected its code. Some unmerged pull requests receive comments or reviews from people. Others have no recorded human response. We study this difference and its effect on reported rejection reasons.**

We call a comment or review by a likely human account a human trace. A pull request is abandoned when it is unmerged, is closed or has been open and silent for at least 60 days, and has no human trace during its observation window. This definition measures recorded activity, not all human attention. We build a rule-based filter and check it against hand labels for 300 pull requests. The hand-labelled sample contained no false abandonment labels, and weighted recall was 0.986.

We apply the filter to 71,677 pull requests from six AI coding agents in public repositories with more than 100 stars. Of the 19,769 unmerged pull requests that were closed, or open and silent for at least 60 days, 12,850 (65.0%) were abandoned. We also re-analyse a published study of rejection reasons. It labelled 306 unmerged pull requests by hand. We matched 302 of them to our data, and 168 (55.6%) were abandoned. Setting them aside raises the share labelled Implementation issues from 9.9% to 22.4%. The same 30 pull requests carry this label, but the total falls from 302 to 134.

These results suggest that unmerged pull requests with and without human traces are worth reporting separately. We recommend that studies also report category shares both with and without abandoned pull requests.

***Index Terms*—Software Development, AI Coding Agents, Pull Requests, Collaboration, Code Review, SDLC**

I. Introduction

We see more and more AI coding agents writing code and opening pull requests. Agents such as OpenAI Codex, GitHub Copilot, Devin, Cursor, Claude Code, and Google Jules now open thousands of them each month on public repositories alone [1], [2]. This raises an obvious question. How good is the code written by these AI coding agents? And is their work actually adding value to the software projects. The field answers this with the merge rate, which is the share of the agent's pull requests that were merged into the project [1]. Agents are ranked by this number, and the ranking is discussed as if it described code quality.

We argue that the merge rate alone cannot answer that question. Unmerged pull requests include two different groups. A person may have commented on or reviewed it. Or there is no recorded comment or review by a person: a bot closed it, a person closed it without a comment, or it stayed open with no activity. Several studies of rejection reasons count closed pull requests of this kind as rejections [3]–[5]. Yet the absence of a human response does not show that a person judged the code and rejected it.

We call the second case abandonment. An abandoned pull request was not merged, is closed or has been silent for at least 60 days, and has no human trace during its observation window. A human trace is a recorded comment or review by a likely human account. Silence means no recorded activity from either people or bots. Mixing abandoned pull requests with those that were engaged but not merged has a clear cost. Suppose one agent sends its pull requests to busy, well-staffed projects, and another one sends them to quiet ones. The first agent will most likely have the higher merge rate, even if the code is identical. The reported difference would describe where the agents were deployed, not the quality of code they produced.

We ask two questions.

RQ1: How much of the unmerged agentic pull request population is abandoned, with no human trace?

RQ2: How do reported category shares change when we separate unmerged pull requests with and without human traces?

This paper makes two contributions. The first is a way to measure the problem. We define abandonment and build a filter that finds it, using simple automation rules and no human judgement. We then validate the filter against a sample that we read by hand, and report how often it is right. The rules are public, so other researchers can reuse or modify them.

Second, we show how this distinction changes a published result. Abujadallah et al. [5] labelled reasons for rejection in a sample of 306 closed, unmerged pull requests. We match 302 of these to our data. Of those, 55.6% have no human trace. When we set them aside, the share labelled Implementation issues rises from 9.9% to 22.4%. We keep the original labels and remove only the abandoned pull requests. Reporting results for both groups helps readers see how much the absence of human traces affects the findings.

II. Related work

A. Why agentic pull requests are not merged

The AIDev dataset [1], [2] collects pull requests that were written by AI coding agents on GitHub. It has made a small

field possible in about a year, and most of the studies ask the question: why are these pull requests not merged?

Three studies answer it by hand. One sorts 600 failures drawn from about 33,000 pull requests into groups [3]. A second study looked at 8,106 unmerged fix-related pull requests and reports twelve reasons [6]. A third labels 306 unmerged pull requests and reports fourteen rejection categories [5]. Their abstract states that 46.41% of pull requests written by AI coding agents are rejected. Other studies reach the same question through either a borrowed taxonomy [4], a language model [7], or association rule mining [8]. All of them define failure in the same way: closed and not merged.

Two details in the third study show the problem with this clearly. First, 17.3% of its rejections are inactivity, meaning the pull request was closed automatically after a period without interaction. The study labels these next to categories such as "incorrect fix". Second, the authors could not label 49.3% of the pull requests in their sample dataset. They report this as a limitation, saying that there was not enough context to say why those pull requests were closed. That unlabelled half is the population that we study. We show in Section IV that there was no context to read because most of these pull requests have no human trace.

Looking at other studies, we can see the same situation occurring multiple times. The largest rejection category in the first study is "abandoned or not reviewed". They defined it as a pull request closed with no meaningful human interaction, and then placed in one table next to build failures and wrong implementations [3]. "Inactive" is the single largest reason in the taxonomy study [4]. And the language model study reports that only 18.1% of its rejected pull requests carry any review comment, and does not return to the point [7]. The evidence for our argument is already in these papers. What is missing is a measure.

Two recent studies question the merge rate itself. The first reads 717 agentic pull requests by hand to recover why each was closed or merged [9]. Of the rejections, 35.7% are clear agent failures, 31.2% are driven by workflow, and 33.1% carry no observable reason at all. The conclusion of their study was that the outcome of a pull request reflects the review environment as much as the quality of the code. Two things differ from our study. The first is method: four annotators reading a stratified sample, releasing a coding guide rather than a tool, and closing with a call for future work to detect these cases automatically at scale. The second matters more. Before analysis, the study removes 1,249 pull requests that received no human comment, because they have no human decision to label the result to. Those pull requests are our subject. We measure the group that their study sets aside.

The second classifies review comments with regular expressions, separating comments that guide an AI coding agent from comments that review the code [10]. It reports that 61.38% of agentic pull requests in AIDev carry no recorded review, and that 84.0% carry either none or one written only by other AI coding agents. Their unit is the comment and the question is whether any human trace exists. They found that inside repositories that receive both agentic and human pull requests, the share showing human participation is almost the same for both, 30.1% against 30.8%. Recorded human participation appears to depend more on the project than on who wrote the patch.

Neither study reports category shares for unmerged pull requests with and without human traces, and neither recomputes a published result. We do both, and we release the tool that does it.

B. Abandonment in human pull requests

Abandonment in human pull requests is quite an old term, and it means something else in published studies. In work on human pull requests, an abandoned pull request is one where the *contributor* stopped answering after review comments arrived [11], [12]. Our definition of abandonment is different: the pull request has no human trace at all. Their abandonment requires engagement to have happened; ours requires that it did not. We keep using the same word because it fits a pull request that never got a human trace, but the two constructs should not be confused.

Those studies still support our reading. A survey of 619 contributors found that the most common reason for walking away was no answer from the maintainers [11]. A study of 4,450 abandoned pull requests found that features of the review process, the contributor and the project predict abandonment better than features of the pull request itself [12]. Contributors put it plainly: being ignored is worse than being turned down [13]. What changes with AI coding agents is scale and placement. There are more pull requests per maintainer, and agents are not pointed at projects at random.

C. Prediction, bots, and trusting absence

Work on mining GitHub warns that platform state fields do not always show the real development process. Some pull requests are merged or handled outside any visible GitHub record [14]. Predicting the result of a pull request at creation time is a mature line of work. Merge decisions have been modelled with random forests, where the strongest predictor feature is whether the change touches recently modified code [15]; acceptance with mixed-effects logistic regression, where the strongest single predictor is the social distance between the submitter and the person who closes the request [16]; and latency with linear regression [17]; and later with gradient boosting under temporal and cross-project splits [18]. The strongest predictors in these models are mostly not properties of the pull request itself. They describe the project, the reviewer, and the history between them. Our contribution is not another outcome model. It is the observation that a measure resting on those factors is read by many as a measure of code quality.

Our construct depends on telling a person from a machine, and on trusting absence. Both are known problems. Bot detection on GitHub repositories has been studied before [19]. Their ground truth dataset of 5,000 GitHub accounts found that 527 of them are bots, and they report that matching "bot"

in an account name finds only 74.7% of them while wrongly flagging 3.8% of humans. What makes this harder is that many active projects do not do all of their work on GitHub [14]. Bots make this harder instead of easier: adopting a code review bot raises merges and lowers the number of human comments at the same time [20].

III. Methodology

A. Where the data comes from

We use AIDev-pop, the subset of the AIDev dataset that covers repositories with more than 100 stars. The unit of analysis is the pull request. AIDev-pop holds 71,677 agentic pull requests. We use four tables. `pull_request` gives the pull request details. `pr_comments`, `pr_reviews`, and `pr_review_comments` give the comments and reviews that we link to each pull request. Account names and account types come from these three event tables. The data covers six agents: GitHub Copilot, OpenAI Codex, Devin, Cursor, Claude Code, and Google Jules. We take the agent label from AIDev and do not re-derive it. Our population is therefore whatever the AIDev detection identifies as agentic. Agent use that leaves no trace, such as a developer who pastes agent output into a commit of their own, is outside our sample, and we cannot say how large that group is.

One property of this population matters for the rest of the paper, because it is easy to assume the opposite. An agentic pull request is not always opened by the agent. The account that opens it depends on the agent: for Copilot, Devin, and Google Jules it is the platform's own account, while for Claude Code, Cursor, and OpenAI Codex it is the account of the person who ran the agent. Our sample therefore mixes both cases, and the author of a pull request is often a human account. This is a property of the platforms rather than of our sampling.

AIDev also has a larger pull request table that covers repositories of every size. That table is useful for broad descriptive counts, but it is not enough for our study. Our construct is defined by something that is missing, because a pull request counts as abandoned when no human engagement is found. To claim that an event is absent, we need a data source in which that event would have been recorded if it had happened. AIDev-pop is the only part of the dataset that carries the comment and reviews table, so it is the only part where silence can be read as evidence.

B. What counts as abandoned

We define abandonment as an unmerged pull request that receives no recorded likely human comment or review activity during its observation window. We call such activity a human trace. A human trace does not show that a person decided against the change. It only shows that a likely human account wrote something. A person may also close a pull request without a comment. We do not count this as engagement, because a close gives no written reason. So the label describes what the data records, not what people saw or thought.

The observation window depends on the state of the pull request on the snapshot date. The snapshot date is 29 October 2025, the last event in our data. For a closed pull request, the window runs from the moment it was opened until one hour after it was closed. For a pull request that is still open, the window runs from the moment it was opened until the snapshot date:

```
closed: [created_at, closed_at + 1h]
open:   [created_at, snapshot]
```

A closed pull request needs no waiting period. Its outcome is fixed when it closes, so we can label it at once. This also means that a closed pull request can be labelled abandoned even when it was open for only a short time. An open pull request is different. Its outcome is not fixed yet, and a person may still answer it. So we only judge an open pull request when it has been silent for at least 60 days before the snapshot date. Silent means that the pull request has no recorded event at all in that period, from a person or from a bot. For a pull request with no events, we use the time it was created. With our snapshot date, the last event of a silent pull request is on or before 30 August 2025:

```
silent: last_event <= snapshot − 60 days
```

The length of this silence is a choice, so it needs a reason. Two limits decide it. The lower limit comes from how fast people usually answer a pull request, when they answer it at all. Yu et al. [17] report that median time to the first comment is 16 minutes and the median time to close is 11.2 hours. So a period of several weeks is already much longer than the time a first human answer usually needs. The upper limit comes from our data. Earlier studies on abandonment in human pull requests are more careful. Khatoonabadi et al. [12] only consider a pull request as possibly abandoned after six months with no activity from the contributor. We cannot use such a long period here. The AIDev data was collected over a short time, so with six months of silence we could only judge the oldest open pull requests. Also, the share of each agent changes quickly, so this would not be a neutral choice. Devin writes about 98% of the oldest pull requests, while Copilot writes 56.9% of the pull requests created in September. A long period would make us more sure that there was really no answer, but the open pull requests that we could judge would then come mostly from one agent. For this reason, we see 60 days as a period that is long enough to make a first human answer very likely, and short enough to keep a large part of the open pull requests in the analysis. We do not see it as a limit that has a neutral meaning by itself. A different length would change which open pull requests we can judge, so it could change the reported rate. The limit only affects open pull requests. Closed pull requests do not depend on it, and they hold 87.0% of all abandonment.

A pull request is abandoned if one of two conditions is true. A closed pull request is abandoned when it was not merged and has no likely human engagement inside its window. An open pull request is abandoned when it has been silent for

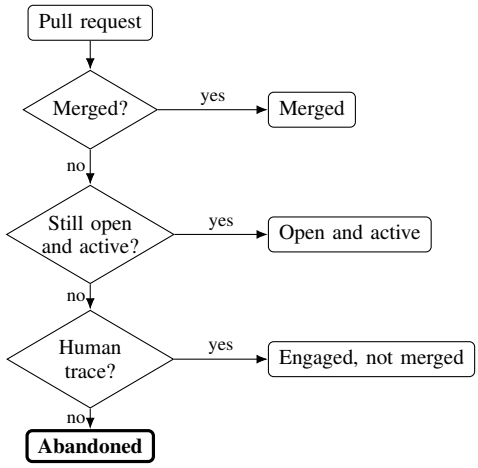


Fig. 1. How every pull request gets exactly one label. *Active* means some recorded event, from a person or a bot, in the sixty days before the snapshot date. A *human trace* is a comment in the thread, a submitted review of any state, or a comment on the diff, written by an account that passes the bot filter and falling inside the observation window. A merged pull request counts as engaged by definition, so it is never abandoned. A close does not count, because it gives no written reason.

at least 60 days and has no likely human engagement inside its window. The extra hour in the window of a closed pull request matters. A maintainer often writes "Closing this, it is already fixed in #123" and closes the pull request in one action. Without the extra hour we would throw that comment away and call the pull request abandoned, when a person clearly read it. This affects 260 pull requests in our data, and the median gap between the close and the comment is 22 seconds. So the final target is:

```
is_abandoned = is_closed_abandoned
            OR is_open_abandoned
```

We give every pull request exactly one of four labels. Fig. 1 shows the three tests that decide which, in the order they are applied. A merged pull request was merged. The abandoned and engaged, not merged labels need a pull request to be decided: closed, or open and silent for 60 days. This is a rule of our analysis, not a claim that a person decided. An abandoned pull request is decided, not merged, and has no human trace. An engaged, not merged pull request is decided, not merged, and has a human trace. This includes 804 open pull requests that are silent but carry a human trace. An open and active pull request has not been silent long enough to judge. It gets this label first, even if it has a human trace.

C. What counts as a human trace

To define if a human left a trace, we collect engagement events from three sources: pull request comments, reviews, and inline review comments. Together, these sources cover the main ways in which a person can respond to a pull request on GitHub. Pull request comments are written in the general discussion of the pull request. Reviews are formal responses in which a reviewer can approve the change, ask for changes, or simply leave a comment. Inline review comments are attached to specific lines of code in the change. By using all three sources, we also count people who only looked at the code and left short notes on it, and not only people who took part in the general discussion.

An event counts as human when three conditions are true. First, GitHub reports the actor as a user account and not as a bot account. On GitHub, accounts that belong to GitHub Apps are marked as bots, and their names usually end with `[bot]`, as in `dependabot[bot]`. But this field alone is not enough, because some automation accounts are registered as normal users. These accounts look like personal accounts, but scripts or services control them. Second, the account name is not on a list of known automation accounts. We made this list by looking at the accounts that create the most events in our data. Third, the event happens inside the observation window that we defined above. For a closed pull request, this means between its creation and one hour after it was closed. For an open pull request, it means any time between its creation and the snapshot date.

Finding bots is a research problem by itself, and there are trained classifiers that can separate bot accounts from human accounts in GitHub comment data [19]. For example, Golzadeh et al. built such a classifier, called BoDeGHa, which uses patterns in the comments that an account writes. Our list only comes close to that standard and does not apply it. We did this because we wanted a rule that we could check by hand for the accounts that create most of the events in our sample. A list that we read ourselves is simple and easy to check, but it can still miss automation accounts that create only a few events. Mistakes in both directions can affect the abandonment rate. If a bot account is counted as human, a pull request may look engaged even though no person responded to it, and the abandonment rate becomes lower than it really is. If a human account is wrongly treated as a bot, the rate becomes higher than it really is.

So the construct measures the lack of recorded comments or reviews that were made by humans. It does not measure the lack of all possible human attention. A person may read a pull request without writing anything, close it without a comment, react with an emoji, add a label, or discuss it outside GitHub, for example in a chat channel. For Claude Code, Cursor, and OpenAI Codex, the person who opens the pull request may also have seen the change in the agent's own tool. Our three sources count none of this, so an abandoned pull request may still have been seen by a person.

D. How we checked the labels by hand

Our definition rests on absence. We call a pull request abandoned when we find no sign of a human on it. A definition like this is hard to check, because a lack of evidence can have two causes. Either no person responded to the pull request, or a person did respond and our pipeline missed it. To test this, we drew a gold sample of 300 pull requests, with 75 pull requests for each of the four labels. Because the sample was drawn with a fixed seed, anyone who uses our package can draw exactly the same 300 pull requests.

TABLE I
THE GOLD SAMPLE. ROWS GIVE THE LABEL OUR RULES PRODUCED. COLUMNS GIVE THE LABEL THE AUTHOR GAVE AFTER READING THE PULL REQUEST ON GITHUB.COM. WE DREW 75 PULL REQUESTS FOR EACH OF OUR FOUR LABELS. THE DIAGONAL IS AGREEMENT. *Eng., n.m.* IS SHORT FOR ENGAGED, NOT MERGED.

	Read by hand				
Our label	Merged	Aband.	Eng., n.m.	Open	Total
Merged	75	0	0	0	75
Abandoned	0	75	0	0	75
Engaged, not merged	2	2	71	0	75
Open and active	0	0	0	75	75
Total	77	77	71	75	300

We wrote down the rules for reading the pull requests before any labelling started. Because the rules were fixed first, we could not change them later to fit the cases we saw. One author then opened each pull request on github.com and gave it a label. Three rules keep the check honest. First, judge the pull request as it was on the snapshot date, which means ignoring about ten months of later activity. The pipeline only saw data up to this date, so a comment written later could make the pipeline look wrong when it was not. Second, do not read the pipeline's own columns before deciding. Otherwise, the check would only compare the pipeline with itself, because a person who has already seen an answer tends to agree with it. Third, write "unclear" instead of guessing. This keeps doubtful cases apart from real decisions. The check tests whether the automated filter and a person applying the same definition produce the same labels

IV. RESULTS

Our snapshot holds 71,677 pull requests [1], [2]. They come from six agents across 6,673 repositories. They were opened between 24 December 2024 and 24 October 2025. The last event in the data is 29 October 2025, and we use that as the snapshot date.

A. How well our rules match the hand labels

Our label rests on absence, so we checked it by hand. We drew 300 pull requests, 75 for each of our four labels, and read each one on github.com as it stood on the snapshot date. Section III gives the rules we followed. The hand label and the label from our rules agree on 296 of the 300. Precision is the share of pull requests labelled abandoned by our rules that really are abandoned. Recall is the share of really abandoned pull requests that our rules find. Precision is 1.000 (95% interval 0.952 to 1.000). Each sampled pull request stands for a different number of real ones, so we weight the sample to compute recall. Recall is 0.986. Table I gives the full comparison.

All four disagreements sit in one row. Every pull request that our rules called abandoned was also abandoned when read by hand. So our rules never gave the abandoned label by mistake. Two pull requests that we labelled engaged, not merged were in fact abandoned. This is where the missing recall comes from. The other two had in fact been merged, so they do not touch the abandoned label. Weighted, the two missed pull requests stand for about 185. If we correct all four errors, the abandoned share for RQ1 rises from 65.0% to about 66.6%. So in this sample our rules undercount abandonment a little and never overcount it.

TABLE II
THE FOUR LABELS ACROSS 71,677 PULL REQUESTS.

Label	Pull requests	Share
Merged	48,451	67.6%
Abandoned	12,850	17.9%
Engaged, not merged	6,919	9.7%
Open and active	3,457	4.8%

TABLE III
ABANDONMENT PER AGENT. *Merged* IS THE MERGE RATE. *Unmerged* COUNTS THE PULL REQUESTS THAT WERE NOT MERGED AND WERE DECIDED.

Agent	PRs	Merged	Unmerged	Abandoned	Share
OpenAI Codex	35,521	82.9%	5,704	4,775	83.7%
Copilot	23,496	50.8%	9,056	4,682	51.7%
Devin	6,172	55.1%	2,655	1,701	64.1%
Cursor	4,059	52.4%	1,711	1,354	79.1%
Claude Code	1,942	65.4%	487	211	43.3%
Google Jules	487	58.1%	156	127	81.4%
All	71,677	67.6%	19,769	12,850	65.0%

B. RQ1: How many unmerged pull requests are abandoned

We give every pull request exactly one of four labels. A pull request is decided when it was merged, when it was closed, or when it has been open and silent for at least 60 days. The rest are still open and still moving, so we cannot judge them yet. Applying the rules of Section III gives the counts in Table II.

19,769 pull requests were not merged and were decided. A merge rate counts all of them as not merged. Of these, 12,850, or 65.0%, have no human trace. The other 6,919, or 35.0%, have one. This is our answer to RQ1. So this group is not one outcome but two. Abandonment is the larger of the two.

Two groups make up the abandoned total. The first group is closed and not merged. There are 17,294 such pull requests, and 11,179 of them have no human trace. The second group is still open and has been silent for at least 60 days. There are 2,475 such pull requests, and 1,671 of them have no human trace. The closed group is 15.6% of all pull requests, and the open group is 2.3%. So most abandonment sits in the closed group, which holds 87.0% of it.

The human filter does real work in both groups. It removes 35.4% of the closed group and 32.5% of the open group. This matters, because abandonment is not simply a new name for "closed and not merged". More than a third of the pull requests that end that way have a person on them, and our rules keep those apart.

The rate is not the same for every agent. Table III splits it. The last column runs from 43.3% to 83.7%. So the correction we propose is not a fixed amount that can be taken off every

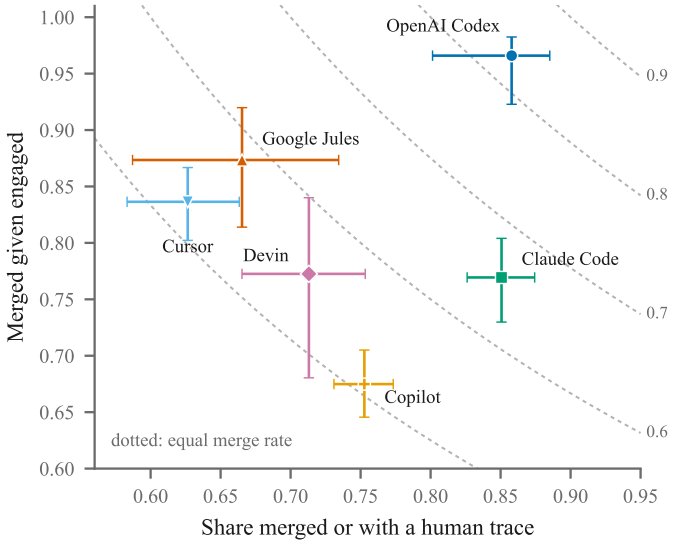


Fig. 2. The merge rate of each agent, split in two. The x-axis is the share of pull requests that were engaged, which means merged or with a human trace. Being open and active alone does not count. The y-axis is the share of engaged pull requests that were merged. The merge rate is the product of the two, and each dotted line joins the points with the same merge rate. The bars are 95% intervals from a bootstrap that resamples whole repositories.

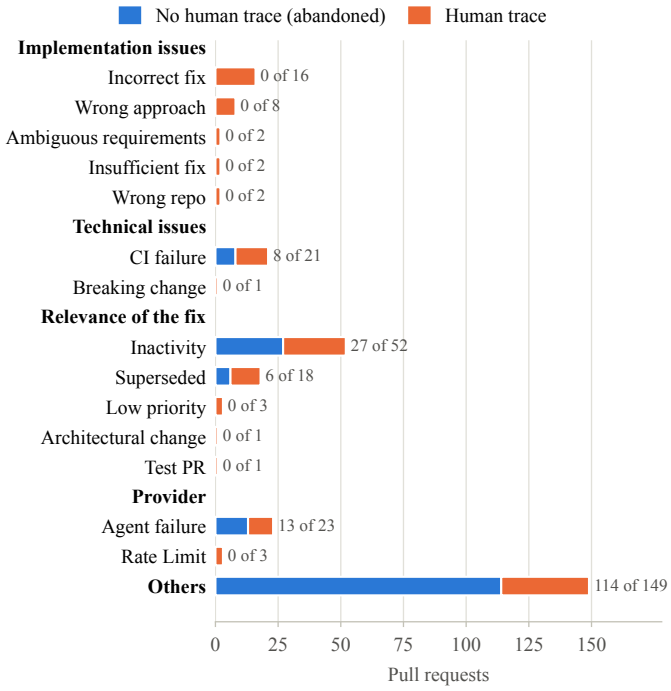


Fig. 3. The pull requests of Abujadallah et al. [5], split by our label. Each row is one of their categories, and bold rows are their high-level groups. The number after each bar gives the abandoned pull requests out of all pull requests in that row. We show the 302 of their 306 pull requests that are in AIDev.

agent. It changes some agents much more than others, and a study that pools agents cannot see this.

Codex and Copilot show the point best, and they are the two largest agents in the data. Codex has the highest merge rate, 82.9%. Its unmerged group is small, only 5,704 pull requests, but 83.7% of that group has no human trace. Copilot has a much lower merge rate, 50.8%, and a much larger unmerged group of 9,056 pull requests. Only 51.7% of that group is abandoned. So almost half of Copilot's decided, unmerged pull requests carry a human trace.

Fig. 2 shows the same split in a second way. We call a pull request engaged when it was merged or carries a human trace. Merging counts by definition, whether a person or an automatic rule did it. Open and active pull requests with a human trace also count as engaged here. Every merge rate is then the product of two shares. The first is the share of pull requests that are engaged. The second is the share of engaged pull requests that were merged. Neither tells us why a pull request was not merged. We compute both over all pull requests of an agent, so their product is exactly the merge rate in Table III.

The bars in Fig. 2 are 95% intervals from a cluster bootstrap. A bootstrap draws many new samples from the data and computes the shares again for each one. We draw whole repositories and not single pull requests, because pull requests from the same repository are not independent. We use 2,000 draws. The bars are wide for Google Jules, which has only 487 pull requests, and for Devin on the y-axis.

Cursor and Copilot show why the split matters. Their merge rates are almost the same, 52.4% and 50.8%, so they sit close to the same dotted line. But they sit at opposite ends

TABLE IV

CATEGORY SHARES BEFORE AND AFTER WE REMOVE THE ABANDONED PULL REQUESTS. *Before* IS OVER THE 302 PULL REQUESTS WE CAN FIND IN AIDEV. *After* IS OVER THE 134 THAT REMAIN ONCE THE 168 ABANDONED ONES ARE REMOVED. *Change* IS IN PERCENTAGE POINTS. *Abandoned* IS THE SHARE OF EACH CATEGORY THAT IS ABANDONED.

Category	Before	After	Change	Abandoned
Others (could not label)	49.3%	26.1%	−23.2	76.5%
Relevance of the fix	24.8%	31.3%	+6.5	44.0%
Implementation issues	9.9%	22.4%	+12.5	0.0%
Provider	8.6%	9.7%	+1.1	50.0%
Technical issues	7.3%	10.4%	+3.1	36.4%

of it. Only 62.7% of the Cursor pull requests were engaged, and 83.6% of those were merged. For Copilot, 75.3% were engaged, and only 67.5% of those were merged. Their intervals do not overlap on either axis. Claude Code and Codex show the other case. Almost the same share of their pull requests was engaged, 85.1% and 85.8%. But 96.6% of the engaged Codex pull requests were merged, against 76.9% for Claude Code.

C. RQ2: What changes in a published study

We now apply our label to a published result. Abujadallah et al. [5] drew a random sample of 306 pull requests from AIDev. Each of them proposes a fix, and each was closed and not merged. They come from Copilot, Devin, Cursor, and Claude Code. Two raters read every pull request and gave it one reason for rejection. Their agreement was 0.605 on Cohen's kappa,

and a third author settled the 93 cases where they disagreed. The result is fourteen sub-categories in four high-level groups. A fifth group, Others, holds 151 pull requests, or 49.3%. For these, the raters found no clear reason for rejection.

The study publishes its label for every pull request, so we can join it to ours. We match on the repository and the pull request number. 302 of the 306 pull requests are in AIDev. The other four are not, so our rules cannot label them, and we leave them out.

All 302 pull requests were not merged and were decided, so each one is either abandoned or engaged, not merged. 168 of them, or 55.6%, are abandoned. This share is lower than the 65.0% in Section IV-B, mainly because the study leaves out OpenAI Codex (Table III).

Fig. 3 splits the 302 pull requests by their categories. No pull request labelled Implementation issues is abandoned. Our rules find a human trace on all 30 of them. A reason such as an incorrect fix or a wrong approach usually needs a person to write it in a comment. Where the raters found no reason, our rules mostly find abandonment. 114 of the 149 pull requests in Others have no human trace, which is 76.5%.

The other four sub-categories are mixed. Inactivity has 27 abandoned pull requests out of 52, Agent failure 13 out of 23, CI failure 8 out of 21, and Superseded 6 out of 18. These reasons can come from messages that a machine writes. Examples are a bot that closes old pull requests, a failed CI run, or a message from the agent.

Removing the abandoned pull requests changes the picture. 134 pull requests remain, and each of them has a human trace. Table IV gives the full numbers. The share of pull requests labelled Implementation issues rises from 9.9% to 22.4%. Others falls from 49.3% to 26.1%. Relevance of the fix, at 31.3%, becomes the largest group. Technical issues rise from 7.3% to 10.4%, and Provider from 8.6% to 9.7%.

We do not claim that the published labels are wrong. Each label describes what is visible on the pull request. But a pull request with no human trace still gets a category, usually Others or a reason that a machine wrote, such as Inactivity. Such a category does not show that a person judged the change.

So separating unmerged pull requests with and without human traces changes the reported category shares. More than half of the 302 pull requests have no human trace. Once we set them aside, the share of pull requests labelled Implementation issues more than doubles. This is our answer to RQ2.

V. Discussion

A. What merge rate can tell us

Merge rate shows what part of an agent's pull requests actually gets into the codebase. This is useful if we want to know how much of the proposed work a project accepts. But it does not tell us much about why the rest was not merged. Some unmerged pull requests have a human trace. Others have none, or at least none that was recorded. The merge rate counts both groups the same way, and that hides a real difference between them.

Cursor and Copilot show this well. Their merge rates are almost the same, but Fig. 2 shows that they get there in different ways (Section IV-B). If we ranked the agents only by merge rate, we would not see this difference. Looking at the two shares separately lets us ask two questions. Which pull requests are engaged? And among the engaged ones, which were merged?

Most unmerged pull requests meet our definition of abandonment. They have no human trace within their observation window, so no person recorded a reason why they were not merged. We should not take this silence to mean the maintainers thought the code was bad. It does not show that the code was good, either. Our measure can only see that comments and reviews are missing.

The merge rate among engaged pull requests has limits too. A human trace does not prove that someone read the code and turned it down. A change can be correct and still be closed, for example because someone already did the same work. Agents also work on different tasks in different repositories, and this can change both shares (Section VI). So we mainly use this split to describe what happens to agent contributions. It does not tell us much about code quality.

B. Why the group being studied matters

RQ2 shows that the set of pull requests we count changes how we read the reasons they were left unmerged. In Table IV, Implementation issues rise from 9.9% to 22.4%. We did not find any new implementation problems, and we did not change the published labels. The same 30 cases are simply divided by 134 pull requests instead of 302. So implementation problems now make up a larger share of the unmerged pull requests with a human trace. This tells us nothing new about the quality of the code the agents wrote.

The two samples answer different questions. The published sample holds 302 closed, unmerged fixes, with or without a human trace. The filtered sample keeps only the 134 that have a human trace. We think both are useful. A failed build can point to a real problem even if nobody comments on it. So one pull request can be abandoned and still contain a technical problem.

The Others category also gets smaller. Most pull requests in this category, 76.5%, have no human trace. For these, the raters may have had nothing to read. More detailed categories would not help, because nobody wrote a reason down. But Others still holds 26.1% of the filtered sample, so even with a human trace, we cannot always explain why a pull request was closed.

C. What future studies should report

Future studies should keep two things apart when they report on pull requests: what happened to each pull request, and whether it carries a human trace. For status, that means saying whether it was merged, closed without being merged, open and silent, or open and active. A human trace is a separate question: did the pull request get any comment or review by a likely human account? Researchers also need to explain how

they decided which accounts belong to humans, and how long they watched each pull request. Abandonment shares should be reported next to merge rates, for each agent, because they differ a lot: from 43.3% to 83.7% in our data (Table III). And a percentage on its own is not enough. It should come with the count behind it and a clear note on which pull requests it covers.

When a study looks at why pull requests were never merged, it should also say where the evidence for each reason came from. A reply from a reviewer is one kind of evidence. A bot message or a test result is another. Technical issues should not drop out of the data just because nobody commented. Papers should report results for the full sample and for the pull requests with a human trace. Then readers can see how much that choice changes the findings.

Future work could compare agents inside the same repositories and on similar tasks. That would show how much of the difference comes from where the agents are used. Studies could also repeat the analysis with a shorter or longer silence limit than our 60 days. This limit only affects open pull requests. Closed pull requests hold 87.0% of all abandonment, and they do not depend on it.

VI. Threats to validity

Construct validity. Our labels measure visible engagement. The rules flag unmerged pull requests that have no recorded comment or review likely made by a human. People may still have read these pull requests or closed them without a comment, and some discussion may have happened elsewhere. A comment is also weaker evidence than it looks. It may come from the author, and even a maintainer's comment does not show that they reviewed or rejected the change. So the labels describe what is visible on the pull request. They do not cover all human attention, and they say nothing about decisions on code quality. Two limits matter most. First, we do not check who closed a pull request. A person may close it without a comment, and we still label it abandoned. Second, for Claude Code, Cursor, and OpenAI Codex, the person who opens the pull request may have seen the change in the agent's own tool. Neither case is an error under our definition, but the label must not be read as proof that no person saw the change. In the other direction, a comment by the person who opened the pull request counts as a human trace, even when it is an instruction to the agent. This makes our abandonment counts lower than a stricter definition that leaves out such instructions.

Internal validity. Missing events or mistakes in bot detection can change the labels. Our manual check of 300 cases supports the rules. But one author checked all 300 using the same definition, so agreement cannot rule out mistakes that the rules and the author share. We found no false abandonment labels in the sample, which does not mean the full dataset has none. The time limit matters too. Some pull requests may get attention after it ends, and a different limit could change the results for some agents more than for others.

External validity. We study six agents in public repositories with more than 100 stars, over one collection period. If AIDev does not detect an agent's use, that use is not in our sample. Each agent works in its own mix of projects and tasks, so comparing their rates does not isolate how well the agents perform. For RQ2 we use one published sample of unmerged fixes. Separating engagement clearly changes that sample, though the size of the change could be different in other data.

VII. Conclusion

A merge rate sorts pull requests into two groups: merged or not merged. But an unmerged pull request can mean two different things. We studied 71,677 pull requests from six AI coding agents. Of these, 19,769 were not merged and were decided, and 65.0% of them have no human trace. We call them abandoned. An unmerged pull request on its own is not evidence that a person decided against the change. Abandonment shares differ by agent, ranging from 43.3% to 83.7%. Studies should report these shares next to merge rates, for each agent.

We can measure abandonment without human judgement, using a filter built from simple rules over public data. On a hand-checked sample of 300 pull requests, precision on the abandoned label is 1.000 and recall is 0.986. In that sample, we found no false positives for the abandoned label, and the filter missed a handful of abandoned pull requests.

Separating unmerged pull requests with and without human traces also changes the reported category shares. Of the 302 pull requests we matched from the sample of Abujadallah et al. [5], 55.6% have no human trace. If we remove those, the share of pull requests labelled Implementation issues rises from 9.9% to 22.4%, more than double. The labels stay the same. Only the group we count changes.

A merge rate mixes two things: whether a pull request was engaged, and whether an engaged pull request was merged. We think both are worth studying, and they are much easier to study once they are kept apart.

Data availability

All code and data that we used for this paper are available at https://anonymous.4open.science/r/abandoned-SE-paper-8A06/.

References

[1] H. Li, H. Zhang, and A. E. Hassan, "The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering," 2025. [Online]. Available: https://arxiv.org/abs/2507.15003

[2] ——, "Aidev: Studying ai coding agents on github," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ser. MSR '26. ACM, Apr. 2026, p. 1029–1033. [Online]. Available: http://dx.doi.org/10.1145/3793302.3797249

[3] R. Ehsani, S. Pathak, S. Rawal, A. A. Mujahid, M. M. Imran, and P. Chatterjee, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[4] K. Gonzalez and M. E. Mezouar, "When bots get the boot: Understanding pull request rejections in the era of AI coders," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[5] M. Abujadallah, A. Arabat, and M. Sayagh, "Understanding the rejection of fixes generated by agentic pull requests: Insights from the AIDev dataset," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[6] K. Alam, S. Mondal, and B. Roy, "Why are AI agent–involved pull requests (fix-related) remain unmerged? an empirical study," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[7] Y. Wang and Z. Yang, "Do AI-generated pull requests get rejected more? (yes but why?)," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[8] M. C. Strauss and S. Schulze, "Why and when agentic pull requests are (not) accepted: An exploratory study," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[9] S. R. O. Peralta, F. Hoshi, H. Washizaki, N. Ubayashi, I. Kondo, Y. Higo, H. Mukai, N. Yoshida, K. Kusama, H. Tanaka, and Y. Fan, "Why are agentic pull requests merged or rejected? an empirical study," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*, 2026.

[10] K. Duma, P. Wróblewski, J. Bobińska, J. Winiarska, and P. Przymus, "These aren't the reviews you're looking for: How humans review AI-generated pull requests," in *Proceedings of the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2026.

[11] Z. Li, Y. Yu, T. Wang, G. Yin, S. Li, and H. Wang, "Are you still working on this? an empirical study on pull request abandonment," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 2173–2188, 2022.

[12] S. Khatoonabadi, D. E. Costa, R. Abdalkareem, and E. Shihab, "On wasted contributions: Understanding the dynamics of contributor-abandoned pull requests," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–39, 2023.

[13] G. Gousios, M.-A. Storey, and A. Bacchelli, "Work practices and challenges in pull-based development: The contributor's perspective," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 285–296.

[14] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, 2014, pp. 92–101.

[15] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 345–355.

[16] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 356–366.

[17] Y. Yu, H. Wang, V. Filkov, P. Devanbu, and B. Vasilescu, "Wait for it: Determinants of pull request evaluation latency on GitHub," in *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*, 2015, pp. 367–371.

[18] S. Khatoonabadi, A. Abdellatif, D. E. Costa, and E. Shihab, "Predicting the first response latency of maintainers and contributors in pull requests," *IEEE Transactions on Software Engineering*, 2024.

[19] M. Golzadeh, A. Decan, D. Legay, and T. Mens, "A ground-truth dataset and classification model for detecting bots in GitHub issue and PR comments," *Journal of Systems and Software*, vol. 175, p. 110911, 2021.

[20] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Quality gatekeepers: Investigating the effects of code review bots on pull request activities," *Empirical Software Engineering*, vol. 27, no. 5, 2022.